Programação de Sistemas Linux

Aula 10 — Threads e Sincronização

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes

Abertura

Notes

Onde estamos

Aula 9 (feito)

- Processo, PCB e estados
- `fork()`, `exec()`, `wait()`
- Zumbis e órfãos
- *Daemons* e *double fork*

Aula 10 (hoje)

- *Thread vs.* processo
- Criar e esperar threads com pthreads
- **Condição de corrida**
- *Mutex*, semáforos, variáveis de condição
- *Deadlock*

Estrutura da aula

Hoje: teoria e demonstrações.
Próximo encontro: **Laboratório 09**.



Notes

Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Explicar o que threads compartilham e por que são mais baratas que processos
- ✓ Criar, esperar e finalizar threads com `pthreads`
- ✓ Reconhecer uma **condição de corrida** e explicar por que ela é silenciosa
- ✓ Proteger uma seção crítica com **mutex**
- ✓ Coordenar produtor e consumidor com **semáforos**
- ✓ Esperar por uma condição sem desperdiçar CPU, com **variáveis de condição**
- ✓ Reconhecer e evitar um **deadlock**

Regra de hoje

Threads compartilham **tudo** — essa é a vantagem (rápido, barato) **e** o perigo. A segunda metade da aula é inteira sobre como não se machucar com a primeira.



Notes

Thread vs. processo

Notes

Thread: um fluxo de execução dentro do processo

Definição

Uma **thread** é um fluxo de execução independente que roda **dentro** de um processo. Um processo pode ter **várias** threads, todas compartilhando o mesmo espaço de memória.

Compartilhado entre threads	Próprio de cada thread
Código do programa	Pilha (*stack*)
Heap (`malloc`)	Registradores da CPU
Variáveis globais/estáticas	PC (*program counter*)
Descritores de arquivo abertos	*Thread ID* (`pthread_t`)

- A pilha de cada thread é **isolada** — variáveis locais são seguras; tudo o mais é **compartilhado por padrão**

Notes

Por que threads são mais “baratas” que processos

- `fork()` precisa duplicar o espaço de endereçamento — mesmo com *copy-on-write*, ainda envolve criar novas tabelas de páginas
- `pthread_create()` só precisa alocar uma **nova pilha** — o resto (código, heap, globais) já existe e é reaproveitado
- Trocar de contexto entre threads do **mesmo** processo também tende a ser mais rápido do que entre processos diferentes

A outra face da moeda

Processos criados com `fork()` são **isolados**: um não corrompe a memória do outro. Com threads, esse isolamento **não existe** — é por isso que precisamos de tudo o que vem na segunda metade desta aula.



Notes

Criando threads com pthreads

Notes

A API básica

```c
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*rotina)(void *), void *arg);
int pthread_join(pthread_t thread, void **retorno);
void pthread_exit(void *retorno);

```

- `thread`: onde o *ID* da nova thread é gravado
- `attr`: atributos (pilha, prioridade…) — `NULL` usa o padrão
- `rotina`: a função que a thread executa — recebe `void *`, retorna `void *`
- `arg`: o argumento passado para `rotina` (normalmente um `struct`)

Compile sempre com `-pthread`.



Notes

Exemplo mínimo

```c
void *ola(void *arg) {
    int id = *(int *)arg;
    printf("Ola da thread %d\n", id);
    return NULL;              /* equivale a pthread_exit(NULL) */
}

int main(void) {
    pthread_t t;
    int id = 1;
    pthread_create(&t, NULL, ola, &id);
    pthread_join(t, NULL);    /* espera a thread terminar */
    return 0;
}

```

- `pthread_join()` é o `waitpid()` **das threads** — sem ele, o processo pode terminar antes de a thread rodar
- Para devolver um resultado, retorne um ponteiro para memória do **heap**; **nunca** o endereço de uma variável local da thread — a pilha dela deixa de existir quando ela termina



Notes

Dividindo trabalho entre threads

Notes

Padrão: dividir um array em N pedaços

```c
typedef struct {
    int id;
    long long inicio, fim;
} tarefa_t;

void *soma_faixa(void *arg) {
    tarefa_t *t = (tarefa_t *)arg;
    long long soma = 0;
    for (long long i = t->inicio; i < t->fim; i++)
        soma += vetor[i];
    somas_parciais[t->id] = soma;   /* cada thread escreve na SUA
     posicao */
    return NULL;
}

```

- Cada thread escreve num **índice diferente** de `somas_parciais` — sem conflito, porque as posições nunca se sobrepõem
- **Essa é a melhor sincronização: a que não é necessária.** Quando dá para dividir o trabalho sem compartilhar escrita, divida

Notes



Criando e esperando as N threads

```
1 for (int i = 0; i < n_threads; i++) {
2     tarefas[i].id = i;
3     tarefas[i].inicio = i * bloco;
4     tarefas[i].fim = (i == n_threads - 1) ? TAM : (i + 1) * bloco;
5     pthread_create(&threads[i], NULL, soma_faixa, &tarefas[i]);
6 }
7 for (int i = 0; i < n_threads; i++) {
8     pthread_join(threads[i], NULL);
9 }
10
```

- **Dois** laços separados: primeiro cria **todas**, depois espera **todas**
- Juntar os dois num laço só faria as threads rodarem **uma de cada vez** — exatamente o mesmo erro do `waitpid()` no meio do laço de `fork()`, na Aula 9



Notes

Quanto isso acelera, de verdade

Soma de 100 milhões de elementos, em uma máquina com `nproc` = **2**:

```
1 1 thread : speedup 0.99  0.98  0.97     (3 execucoes)
2 2 threads: speedup 1.87  1.78  1.93
3 4 threads: speedup 2.21  1.72  1.96
4 8 threads: speedup 2.38  1.91  1.49
5
```

- Com **1** thread o *speedup* é ≈1 — o mesmo trabalho, mais o custo de criar a thread
- Com **2** threads (= número de núcleos) chega perto de **2x**
- Acima disso **não melhora** — só aumenta a **variação** entre execuções, porque as threads passam a disputar os mesmos 2 núcleos



Notes

Mais threads não é sempre melhor

Regra prática

Para trabalho **limitado por CPU** (como somar um array), o *speedup* tem um teto: o número de núcleos (`nproc`). Criar mais threads que isso não acelera — só adiciona troca de contexto.

- Para trabalho **limitado por I/O** (esperando disco, rede), a regra muda: threads em excesso ainda podem ajudar, porque a maioria está **bloqueada**, não competindo por CPU
- Meça, não presuma: o *speedup* também depende de o problema ser realmente divisível e de o acesso à memória não virar o gargalo



Notes

Condição de corrida

Notes

Quando o compartilhamento vira perigo

```c
long contador = 0;   /* GLOBAL -- compartilhada por todas as threads */

void *incrementa(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        contador++;   /* parece uma linha, mas NAO e' uma operacao
     atomica */
    }
    return NULL;
}

```

- `contador++` vira, na prática, **três** passos: *ler* o valor, *somar* 1, *escrever* de volta
- Se duas threads executam esses três passos **intercalados**, um incremento é **perdido**: as duas leram o mesmo valor antigo



Notes

Vendo a condição de corrida ao vivo

```
$ ./race        # 4 threads x 1.000.000 incrementos cada
Contador final: 1386195
Esperado:       4000000
Diferenca:      2613805 (incrementos perdidos)

$ ./race        # de novo -- o resultado MUDA a cada execucao
Contador final: 2250578
$ ./race
Contador final: 2762342

```

- O resultado **varia** a cada execução — assinatura de uma **condição de corrida**: depende de como o escalonador intercalou as threads naquele instante
- **Nenhum erro, warning ou crash** avisa — o programa simplesmente calcula o valor errado, em silêncio
- Perdemos **mais da metade** dos incrementos. Não é um arredondamento: é o programa estar errado



Notes

Mutex: exclusão mútua

Notes

Seção crítica e exclusão mútua

Definição

Seção crítica é o trecho de código que acessa um recurso compartilhado e que, por isso, **não pode** ser executado por duas threads ao mesmo tempo. Um **mutex** (*mutual exclusion*) é um cadeado que garante isso: só uma thread por vez passa.

- Quem chega e encontra o cadeado fechado **dorme** até ele abrir — não fica queimando CPU
- O mutex tem **dono**: quem trancou é quem deve destrancar
- Regra de ouro: a seção crítica deve ser a **menor possível** — veja o porquê dois slides adiante

Notes

Mutex na prática

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
long contador = 0;

void *incrementa(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&mutex);
        contador++;                    /* secao critica */
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

```

```
$ ./mutex ; ./mutex ; ./mutex
Contador final: 4000000
Contador final: 4000000
Contador final: 4000000

```

- Agora o resultado é **sempre** 4.000.000 — determinístico, execução após execução



Notes

O preço do cadeado

```
$ time ./race        # sem mutex (e errado)
real    0m0.012s
$ time ./mutex       # com mutex (e correto)
real    0m0.288s

```

24 vezes mais lento — e ainda assim é a versão certa

Travar e destravar 4 milhões de vezes custa caro. A saída **não** é remover o mutex: é **trancar menos vezes**. Por exemplo, cada thread soma num contador **local** e, no fim, trava **uma única vez** para somar seu total ao global.

- `pthread_mutex_trylock()`: tenta travar e **volta na hora** se estiver ocupado, em vez de dormir
- `pthread_rwlock_t`: permite **vários leitores** simultâneos, mas só **um escritor** — vale a pena quando as leituras são muito mais frequentes que as escritas



Notes

Semáforos

Notes

Semáforo: um contador, não um cadeado

Definição

Um **semáforo** guarda um **contador** de recursos disponíveis. `sem_wait()` decrementa (e **bloqueia** se já está em zero); `sem_post()` incrementa (e acorda quem estiver esperando).

```c
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 5); /* 0 = threads do mesmo proc.; 5 = valor inicial */
sem_wait(&s);       /* pega um recurso (bloqueia se nao houver) */
sem_post(&s);       /* devolve um recurso */
sem_destroy(&s);
```

- Diferente do mutex, um semáforo **não tem dono**: uma thread pode dar `post` no que outra pegou com `wait`
- Mutex responde "*posso entrar?*"; semáforo responde "**quantos ainda tem?**"

Notes

Produtor–consumidor: o uso clássico

```
1 sem_init(&vagas, 0, TAM);   /* posicoes LIVRES  */
2 sem_init(&itens, 0, 0);     /* posicoes OCUPADAS */
3
```

```
/* produtor */
sem_wait(&vagas);   /* tem vaga? */
pthread_mutex_lock(&mutex);
buffer[idx_prod] = i;
idx_prod = (idx_prod+1) % TAM;
pthread_mutex_unlock(&mutex);
sem_post(&itens);   /* tem item! */
```

```
/* consumidor */
sem_wait(&itens);   /* tem item? */
pthread_mutex_lock(&mutex);
v = buffer[idx_cons];
idx_cons = (idx_cons+1) % TAM;
pthread_mutex_unlock(&mutex);
sem_post(&vagas);   /* liberou vaga! */
```

- **Dois** semáforos (vagas e itens) + **um** mutex: os semáforos controlam **quantos**, o mutex protege o **buffer em si** — e os dois são espelhados: o que um consome, o outro devolve



Notes

Produtor–consumidor em execução

Buffer de **5** posições, produtor mais rápido que o consumidor:

```
1 [PROD] produziu 1 na posicao 0
2         [CONS] consumiu 1 da posicao 0
3 [PROD] produziu 2 na posicao 1
4 [PROD] produziu 3 na posicao 2
5 [PROD] produziu 4 na posicao 3
6 [PROD] produziu 5 na posicao 4
7 [PROD] produziu 6 na posicao 0      <- deu a volta no buffer circular
8         [CONS] consumiu 3 da posicao 2
9 ...
10 Todos os 10 itens foram produzidos e consumidos.
11
```

- O produtor **corre na frente**, enche o buffer e então **bloqueia** em `sem_wait(&vagas)` — sem `sleep`, sem laço de espera, sem combinar nada
- Nenhum item é perdido nem lido duas vezes: o buffer circular **nunca** transborda



Notes

Variáveis de condição

Notes

O problema: esperar sem queimar CPU

```c
/* ERRADO: espera ocupada (busy-wait) */
while (fila_vazia()) {
    /* nao faz nada, so gira... queimando um nucleo inteiro */
}

```

- Funciona, mas **desperdiça** 100% de um núcleo girando à toa
- Pior: se o laço estiver **dentro** de um mutex, ninguém nunca consegue mudar a condição — o programa trava de vez

Definição

Uma **variável de condição** permite **dormir** até que outra thread avise que algo mudou. `pthread_cond_wait()` **solta o mutex e dorme** de forma atômica; ao acordar, **retoma o mutex** antes de voltar.

Notes

O padrão correto — e por que `while`, não `if`

```c
/* quem espera */
pthread_mutex_lock(&m);
while (n_itens == 0) {
    pthread_cond_wait(&tem_item, &m);
}
int v = fila[--n_itens];
pthread_mutex_unlock(&m);
```

```c
/* quem avisa */
pthread_mutex_lock(&m);
fila[n_itens++] = valor;
pthread_cond_signal(&tem_item);
pthread_mutex_unlock(&m);
```

`while`, nunca `if`

Uma thread pode acordar **sem** que a condição seja verdadeira — por um despertar espúrio, ou porque outra thread chegou primeiro e já consumiu o item. Com `while`, ela **reconfere** e volta a dormir. Com `if`, seguiria em frente sobre uma fila vazia.

- `signal` acorda **uma** thread; `broadcast` acorda **todas** as que esperam



Notes

Deadlock

Notes

Deadlock: dois cadeados, duas threads, ordens opostas

```c
/* Thread 1 */
pthread_mutex_lock(&mA);
usleep(100000);
pthread_mutex_lock(&mB);
```

```c
/* Thread 2 -- ordem OPOSTA */
pthread_mutex_lock(&mB);
usleep(100000);
pthread_mutex_lock(&mA);
```

```
$ timeout 3 ./deadlock
[T1] peguei A
[T2] peguei B
[T2] quero A...
[T1] quero B...
$ echo $?
124        <- o timeout teve de matar: o programa travou para sempre

```

- Cada uma segura o que a outra quer e espera para sempre — **nenhuma** das duas vai ceder
- Não há erro, não há crash: o programa simplesmente **para**



Notes

A correção: ordem consistente de travamento

```c
/* Thread 1 */
pthread_mutex_lock(&mA);
pthread_mutex_lock(&mB);
```

```c
/* Thread 2 -- MESMA ordem */
pthread_mutex_lock(&mA);
pthread_mutex_lock(&mB);
```

```
$ ./deadlock_ok
[T2] peguei A ... [T2] peguei B
[T1] peguei A ... [T1] peguei B
Terminou (se voce esta vendo isto, nao houve deadlock)

```

- Se **todas** as threads travam os mutexes **sempre na mesma ordem**, a espera circular é impossível — e sem espera circular, não há deadlock
- Na prática: defina uma ordem global (por endereço, por índice, por nome) e documente-a
- `pthread_mutex_trylock()` é a saída quando a ordem não pode ser garantida: se falhar, solte o que já tem e tente de novo



Notes

Encerramento

Notes

Síntese da aula

Ferramenta	Pergunta que responde
`pthread_create/join`	Como crio um fluxo novo e espero ele terminar?
Dividir em N faixas	Como uso vários núcleos sem precisar sincronizar nada?
Condição de corrida	O que quebra quando duas threads tocam a mesma variável?
Mutex	Como garanto que só uma thread entra na seção crítica?
Semáforo	Como controlo *quantos* recursos ainda há disponíveis?
Variável de condição	Como espero algo acontecer sem queimar CPU?
Deadlock	Por que meu programa parou sem erro nenhum?

Como escolher

Proteger um dado compartilhado → **mutex**. Contar recursos ou coordenar produtor/consumidor → **semáforo**. Esperar uma condição ficar verdadeira → **variável de condição**. E, sempre que possível, **não compartilhar**: dividir o trabalho em faixas independentes dispensa os três.

Notes

Atividade prática — próximo encontro

Laboratório 09 — Threads e Sincronização (2h)

Parte A, presencial:

1. Soma paralela com N threads e medição de *speedup*
2. Reproduzir a condição de corrida e corrigi-la com *mutex*
3. Produtor–consumidor com semáforos e buffer circular

Parte B, extraclasse: fila *thread-safe* com variável de condição, provocar e corrigir um *deadlock*, e um *pool* de threads.

Atenção

No exercício da corrida, rode o programa **várias vezes**: uma execução só não prova nada. O que caracteriza a corrida é o resultado **mudar** de uma execução para a outra.



Notes

Próxima aula

Aula 11 — Comunicação entre Processos (IPC)

- *Pipes* anônimos e FIFOs (*named pipes*)
- Memória compartilhada POSIX: `shm_open`, `mmap`
- Filas de mensagens POSIX
- Como escolher o mecanismo certo para cada cenário

Bibliografia de apoio

- Manuais: `man 7 pthreads`, `man 3 pthread_mutex_lock`, `man 3 sem_wait`, `man 3 pthread_cond_wait`
- The Linux Programming Interface (Michael Kerrisk) — capítulos 29–30
- BUTENHOF, D. R. *Programming with POSIX Threads* — Addison-Wesley



Notes

That’s all folks!

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes

Notes